Лекція 5. Файли та каталоги

Операційна система

Операційна система (ОС, англ. *operating system*, **OS**) — це базовий комплекс програм, що керує апаратною частиною комп’ютера або віртуальної машини, забезпечує управління обчислювальним процесом і організовує взаємодію з користувачем.

Операційна система зазвичай складається з ядра ОС та базового набору прикладних програм. У більшості випадків усі взаємодії користувача з операційною системою здійснюються саме за допомогою цих прикладних програм. Проте в деяких ситуаціях виникає потреба створювати додаткові скрипти для вирішення завдань взаємодії з ОС.

Модулі для роботи з ОС

Python, як й інші мови програмування, надає спеціальні модулі для використання можливостей операційної системи. Проте, великою перевагою модулів у Python є їхня крос-платформність.

Деякі популярні модулі для роботи з ОС у Python:

- sys
- os
- glob
- Shutil
- pathlib

Модулі для роботи з ОС

- Пакет **os** містить функції для отримання інформації про операційну систему, роботи з файлами та каталогами, а також для керування процесами. До його складу входить кілька модулів, серед яких найчастіше використовують модуль **os.path**, призначений для роботи з файлами та каталогами.
- Модуль **sys** містить функції, що дозволяють отримувати дані або змінювати системне оточення інтерпретатора Python.
- Модуль **shutil** (*shell utilities – системні програми командного рядка*) дає змогу виконувати одним викликом операції над великими групами файлів і каталогів.
- Модуль **glob** містить функції для роботи з наборами файлів і каталогів.
- Модуль Pathlib використовується для об’єктно-орієнтованої роботи з файловими шляхами.

Модуль sys та стандартні файли

Під час виконання програм інтерпретатором Python використовуються стандартні файли: stdin, stdout, stderr. Ці файли описані в модулі sys.sys.

- stdin – стандартний файл для введення;
- sys.stdout – стандартний файл для виведення;
- sys.stderr – стандартний файл для виведення інформації про помилки;

Python автоматично відкриває стандартні файли під час запуску програми і так само автоматично закриває їх після її завершення.За замовчуванням stdin пов’язаний з клавіатурою, а stdout і stderr — з екраном. Однак ці зв’язки можна тимчасово змінити, щоб виконувати введення або виведення в інші файли.

Файлова система

Файлова система (скорочено ФС) — це сукупність методів і структур, які використовує операційна система комп’ютера для впорядкування даних на цифрових носіях інформації, а також для контролю вільного простору, призначеного для їх зберігання.

Щоб краще зрозуміти сутність цієї технології, варто ознайомитися з основними принципами, що лежать в її основі. Загалом файлова система є структурованим представленням даних разом із набором метаданих, які їх описують. Вона застосовується до носія під час його форматування. Така структура використовується для всього сховища або для його окремого ізольованого сегмента — розділу диска.

Зазвичай файлова система оперує блоками, а не секторами. Блоки ФС — це групи секторів, які дозволяють оптимізувати адресацію сховища. Сучасні файлові системи здебільшого використовують блоки розміром від 1 до 128 секторів (512–65536 байт). Файли зазвичай зберігаються на початку блоку та займають цілі блоки.

Детальніше про файлову систему можна почитати, наприклад, тут.

Файлова система

У сучасних операційних системах файлова система має ієрархічну структуру каталогів у вигляді дерева або ациклічного графа.

Кожен каталог може містити файли або підкаталоги. Один із каталогів вважається поточним, і його можна змінити на інший.

Файл задається ім’ям файлу (рядком), у кінці якого може бути додатково вказано розширення імені через крапку, яке і визначає тип файлу (.txt, .py, .exe, .pdf).

Файлова система

Щоб визначити розташування файлу, потрібно задати шлях у вигляді послідовності каталогів, починаючи з поточного (відносний шлях) або деякого початкового каталогу (абсолютний шлях).

Каталоги та власне ім’я файлу розділяються символом-роздільником:

- \ - Windows
- / - Unix
- / або : - MacOS

Приклади до файлу file.doc у різних операційних системах:

- C:\Documents\file.doc - Windows
- etc/pub/file.doc - Unix
- Mac HD:Documents:file - MacOS

Робота з файлами та каталогами

- Прикладами типових задач під час роботи з файлами є: перевірка наявності файлу в певному каталозі, копіювання або переміщення файлу між каталогами, видалення файлу, а також отримання інформації про файл.
- Типові задачі під час роботи з каталогами: отримання списку файлів і підкаталогів у каталозі, створення нового каталогу та видалення каталогу.

Модуль os

Функція	Опис
os.listdir(path='.')	Повертає список всіх файлів та підкаталогів у каталозі path, де '.' означає поточний каталог
os.walk(top)	Генератор, що повертає послідовність кортежів для всіх підкаталогів дерева, коренем якого є каталог top. Для кожного підкаталогу (і для самого top) повертає кортеж (dirpath, dirnames, filenames), де dirpath – каталог, dirnames – список його підкаталогів, filenames – список файлів каталогу.
os.getcwd()	Повертає поточний каталог (current working directory)
os.chdir(path)	Робить каталог path поточним
os.mkdir(path)	Створює каталог path
os.rmdir(path)	Видаляє каталог path. Каталог має бути порожнім!
os.remove(path)	Видаляє файл path

Модуль os.path

Функція	Опис
os.path.exists(path)	Перевіряє, чи існує каталог (файл) path.
os.path.getmtime(path)	Повертає дату останньої зміни файлу (каталогу) path.
os.path.getctime(path)	Повертає дату та час створення файлу (каталогу) path.
os.path.getsize(path)	Повертає розмір файлу path.
os.path.isdir(path)	Перевіряє, чи є path каталогом
os.path.isfile(path)	Перевіряє, чи є path файлом.
os.path.normpath(path)	Повертає path у стандартному для ОС вигляді.
os.path.join(path, *paths)	Об’єднує декілька частин шляху та імені файлу paths у єдиний шлях path. При цьому символ-розділювач дефолтний з ОС.
os.path.split(path)	Розбиває шлях на дві частини: каталоги + ім’я файлу.
os.path.splitext(path)	Розбиває шлях на дві частини: каталоги + початок імені файлу та розширення файлу, починаючи з ‘.’

Модуль pathlib

Модуль pathlib був доданий у Python 3.4 (2014 рік) для об’єктно-орієнтованої роботи з файловими шляхами. Він дозволяє працювати з файлами та каталогами через об’єкти Path, замінюючи більшість функцій з os та os.path. pathlib автоматично враховує особливості операційної системи (шляхи для Windows і Unix) і робить код більш читабельним та компактним.

Модуль pathlib

Функція	Опис
`Path(path)`	Створює об’єкт шляху до файлу або каталогу.
`Path.exists()`	Перевіряє, чи існує файл або каталог.
`Path.is_file()`	Перевіряє, чи об’єкт є файлом.
`Path.is_dir()`	Перевіряє, чи об’єкт є каталогом.
`Path.mkdir(parents=False, exist_ok=False)`	Створює каталог. `parents=True` створює проміжні каталоги, `exist_ok=True` не викликає помилку, якщо каталог уже існує.
`Path.iterdir()`	Повертає ітератор по всіх об’єктах (файли та підкаталоги) у каталозі.
`Path.glob(pattern)`	Повертає список об’єктів у каталозі, що відповідають шаблону (наприклад, `*.txt`).
`Path.rglob(pattern)`	Рекурсивно повертає всі об’єкти у каталозі та підкаталогах за шаблоном.
`Path.joinpath(*other)`	Об’єднує шлях з іншими частинами шляху, повертаючи новий `Path`.

Модуль pathlib

Функція	Опис
`Path.name`	Повертає ім’я файлу або каталогу (останню частину шляху).
`Path.stem`	Повертає ім’я файлу без розширення.
`Path.suffix`	Повертає розширення файлу з крапкою (наприклад, `.txt`).
`Path.resolve()`	Повертає абсолютний шлях (розв’язує символічні посилання).
`Path.unlink()`	Видаляє файл.
`Path.rename(target)`	Перейменовує файл або каталог у `target`.
`Path.stat()`	Повертає інформацію про файл (розмір, дату модифікації, права доступу).
`Path.open(mode='r')`	Відкриває файл у вказаному режимі (r, w, rb, wb) і повертає файловий об’єкт.

Набори файлів та каталогів. Модуль glob

Інколи виникають ситуації, коли потрібно одночасно звернутись до декількох файлів, що мають спільне ім’я чи його частину. Це можна зробити з використанням модуля glob.

glob.glob(mask) – повертає список файлів та каталогів, що відповідають масці mask. Маска може містити як звичайні, так й універсальні символи.

Універсальні символи:

- - будь-який підрядок

? – будь-який символ.

Наприклад ‘t?s?.*` відповідає файлам усіх розширень з іменами з чотирьох літер першою з яких є t, а третьою s. Тобто це можуть бути файли task.txt, test.py тощо.

Приклад. Побудова списку каталогів впорядкованого за розміром

У прикладі побудовано програму, яка повертає список підкаталогів (!) певного каталогу разом з їхніми розмірами.

Функція dirsize повертає розмір одного каталогу, а dirslist – список каталогів і їх розміри.

Список упорядковується за спаданням розміру каталогу (від найбільшого за обсягом до найменшого).

Основна частина програми отримує параметри з командного рядка, або з консолі, якщо параметр не було надано.

Розв’язання.

Завдання. Додатково реалізуйте виведення результатів у файл ‘dirslist.txt’, перенаправивши стандартне виведення у цей файл і створивши для нього окремий каталог методами os.

Приклад. Збереження файлів із заданих каталогів - backup

Програма у цьому прикладі здійснює backup файлів із заданих каталогів у каталог backup.

- Функція copyfile копіює один файл з одного каталогу до іншого.
- Функція copydir рекурсивно копіює каталог разом з усіма його підкаталогами у інший каталог.
- Функція getbackupname повертає ім’я нового каталогу, в який будуть збережені дані. Це ім’я буде рядком з поточної дати та часу.
- Функція backupdirectories зберігає файли із вказаних каталогів у каталозі backup.
- Програма передбачає пропуск файлу чи каталогу, якщо його не знайдено, і вивід відповідного повідомлення про виникнення помилки.
- Основна частина програми отримує параметри командного рядка або вводу, якщо параметри не були надані.

Розв’язання (os). Розв’язання (pathlib).

Архівування

Архівування файлів - це їх стиснення для подальшого зберігання. У Python є багато модулів, що відповідають за процеси архівування, найпопулярнішими є zipfile і tarfile.

Вони мають набір доволі схожих (але не цілком ідентичних) функцій для обробки архівованих файлів.

Обробка архівованих файлів включає в себе такі етапи:

- Відкриття у режимі читання або запису
- Маніпуляції: читання або запис
- Закриття

Архівування

Функція/Метод	Опис
tarfile.open(name=None, mode='r')	Відкриває файл name для подальшого стиснення або розкриття згідно з режимом mode. Повертає об'єкт класу TarFile.
tf.add(filename)	Додає файл до архіву tf.
tf.extractall(path='.')	Видобуває всі файли архіву tf у каталог path
tf.close()	Закриває файл tf

Розглянемо деякі функції модуля tarfile, який дозволяє стискати у архів та розпаковувати файли з архівів у різних форматах (gzip, bzip2, lzma).

Архівування

mode	Значення
‘r’	Відкрити архів для читання
‘w:gz’	Відкрити для запису (стиснення) у форматі gzip
‘w:bz2’	Відкрити для запису (стиснення) у форматі bzip2
‘w:xy’	Відкрити для запису (стиснення) у форматі lzma

Розглянемо можливі значення параметра mode у функції open.

Приклад. Backup + Архів

- Необхідно забезпечити збереження даних кількох каталогів. Реалізувати їх backap.
- Для того, щоб запобігти вичерпанню ресурсу носія (наприклад жорсткого диску) потрібно реалізувати архівування всіх backup-версій, крім останньої.
- Модифікуємо попередню програму додавши дві функції: archivesubdirs для архівування підкаталогів та removedir для рекурсивного видалення раніше заархівованого підкаталогу з його усіма файлами.

Розв’язок

Модуль shutil

Масові дії над файлами та каталогами можна виконувати, користуючись модулем shututil.

Цей модуль містить функції для копіювання файлів та каталогів, їх видалення, архівування тощо.

Модуль shutil

Функція	Опис
shutil.copy(scr,dst)	Копіює файл (або каталог) scr. Dst – повне ім’я файлу, включаючи шлях до нього. Якщо dst - каталог, то файл буде скопійовано з тим же ж ім’ям, що і scr.
shutil.copytree(scr,dst)	Копіює каталог scr у каталог dst рекурсивно разом з усіма підкаталогами
shutil.rmtree(path)	Видаляє каталог path рекурсивно з усіма підкаталогами. При цьому каталог може бути і не порожнім.
shutil.make_archive(base_name, format[, root_dir])	Створює архів з ім’ям base_name у форматі format та додає до нього усі файли з каталогу root_dir. Значеннями format можуть бути ‘zip’,’tar’, або ’bztar’
shutil.unpack_archive(filename[, extract_dir])	Розпакуваати усі файли архіву filenameу каталог extract_dir

Завдання

Використовуючи попередній код та можливості модуля shutil реалізувати backup з кількох каталогів.

Q&A